# MPI HOMEWORK (part 1)

# Ping Pong Acquaintance

Let’s imagine that processes are human beings and they wanted to play ping pong getting acquaintance with other guys.

Processor 0 starts the play. He randomly passes a ball to “j” and says himself name.

Processor “i” starts pass a ball to **other** guy “j” saying all previous names in the mentioned order and says himself name. Passing occurs in random way; it means only processor “i” knows who will be next, others don’t know and they always need to be on the look-out. The game ends after N passes. Use the synchronous mode MPI_Ssend to send the data.

# Ping Pong Acquaintance

Also implement a program in which they will send the data with fixed length in order to:

1) Insert timing calls to measure the time taken by all the communications. You will need to time many ping-pong iterations to get a reasonable elapsed time, especially for small message lengths.

2) Investigate how the time taken varies with the size of the message (length of names). You should fill in your results in Table 1. What is the asymptotic bandwidth for large messages?

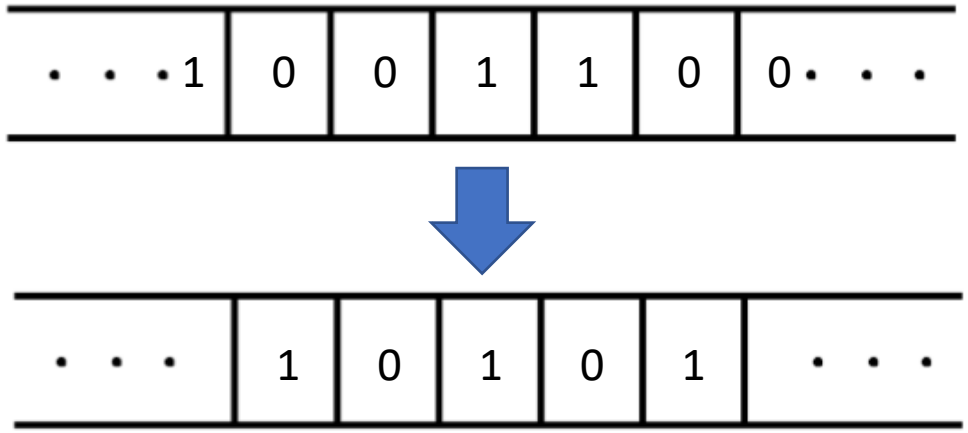
3) Plot a graph of time against message size to determine the latency (i.e. the time taken for a message of zero length); plot a graph of the bandwidth to see how this varies with message size.

The bandwidth and latency are key characteristics of any parallel machine, so it is always instructive to run this ping-pong code on any new computers you may get access to.

| Size (bytes) | # Iterations | Total time (secs) | Time per message | Bandwidth (MB/s) |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

# MPI HOMEWORK (part 2)

# One dimensional cellular automata




Based on the output above the rule could be:

| 111 | 110 | 101 | 100 | 011 | 010 | 001 | 000 |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 |

First: you initialize the one dimensional array with ones and zeros (can do it randomly or not randomly)

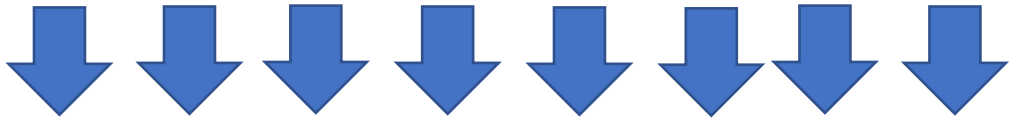
Second: you update the values in the cell according to the cells current value and the left and right neighbors' current values.

So, next state for a cell depends on current state of 3 cells: How many possible update rules there are?

8 possible states of 3 binary cell configuration:

111 110 101 100 011 010 001 000

For every configuration there are two possible updates:

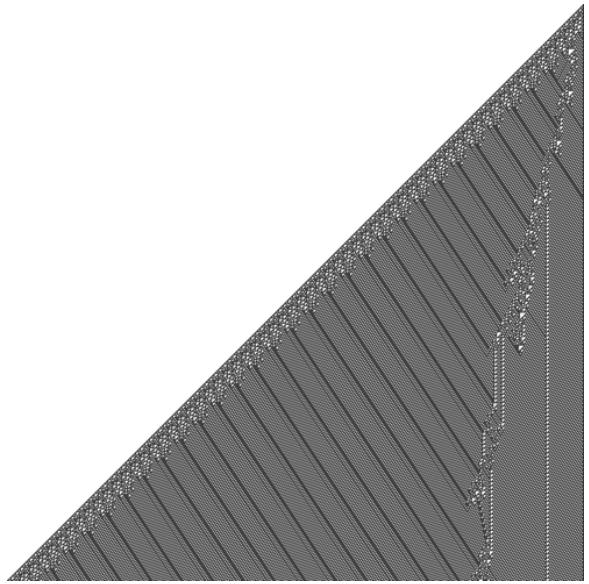


0/1 0/1 0/1 0/1 0/1 0/1 0/1 0/1

In total: 2*2*2*2*2*2*2*2 = 256 rules

# One dimensional cellular automata

Dump every iteration to be able to draw this kind of picture in the end:



This is rule 110
(It's Turing complete!)

Draw pictures for 3 interesting rules and with some boundary and initial conditions of your choice (3 points).

Boundary conditions? Implement both periodic boundary conditions and constant boundary conditions (2 points).

Value updates can be parallelized if you divide computational domain into chunks and allocate it to difference processes (5 points).

Use ghost cells to send and receive values at interfaces between two processes (2 point).

Design a program in such a way that any kind of rule can be easily input into the computations. Wolfram code is a way to standardize rules (5 points).

Make a graph that shows speedup (if size of the array is bigger, then you get greater speedup) (2 points).

DEADLINE 10TH OF MAY

# Homeworks with the star (*)

- You don't have to do them, but these are interesting tasks:

1) Do **parallel** version of the **Schelling** model (see one of the previous presentations).

- we can send you a jupyter-notebook with sequential version, it can help you do parallel version right away.

2) Write a **parallel parser** using MPI that can input a text and output **top10** most used **words** in the text.

- is this task solvable in general? We don't know 😅.

- Sergey wants to know what are the top10 words used in War and Peace.